Computer Games Development CW208
Technical Design Document
Year IV

David Strikaitis
C00283152

13/04/2026

Contents

Contents 2
Technical Design 3
Open GL 4
Structure 4
Game objects 4
input 5
Planet 6
planet editor 6
Save / load 7
shader store 7
model store 7
texture store 8
particles 8
enemy abstract state 8
model partner script 8
surface instancing 8
weapons 9
interaction 9
enemy 9
main menu 9
screen overlay 10

Technical Design

Open GL

The first few weeks of my project was learning how to use Open GL. This is because i never really worked in it properly, this means I had no idea what any of the functions do, how they work, and why they work the way they do this means I researched all the functions in the basic template and played around with how they work

after this I planned the structure of my game

Structure

One thing i dont like about basic game libraries (such as sfml) is how every function needs to somehow pass down functions for instance the render function in sfml a standard structure may have:

```
Game -> Player -> weapon Manager -> weapon -> bullet
```

and this is just one simple structure each of those above need to pass down a render function (for instance, in a standard structure) this means that Game passes render to player, which passes render to weapon manager etc. this is especially bad when something like weapon manager doesn't render anything weapon manager may only hold the weapon (which is rendered), but then weapon manager still needs to implement a render function, even though it doesn't render anything itself I don't like this so I invested a lot of time in creating abstractions for update for instance and I also created abstract script classes which are in abstract game objects

Game objects

I wanted the game objects to function similar to how they work in engines like Unity (one game object, contains scripts) This evolved later to also contain things like tags, for quickly getting the objects of certain tags The game object needed a few specific simple things scripts, transform, and some functions each script needed to implement a start, update, render, and send message function the send message was to try to resemble how unity handles objects calling functions in other game objects in a decoupled way this ended up looking like such Game Object 1 needs to call another game objects function (will use player shoot enemy as example)

player gets all enemies (through tags)

\/

player checks for transform hit scan on enemy

\/

player hits enemy sends message of hit, with damage amount ~ explained next

\/

enemy health controller receives hit

\/

enemy health changes

\/

dies

they send message script also often needs to receive information, this was implemented through the use of std::any std::any is a C++ library function which allows for any parameter type (I use this instead of typenames and such for simplicity and faster compilation)

a vector of an abstract script is central to the design I have This idea mimics how it works in different game engines like Unity My structure allows for the implementation of update functions, or not if it isn't necessary by the script itself

so, each game object looks like this

GameObject

|

|

|- Array of tags

|

|- array of scripts

|

|- Transform

I have abstracted the calling of all the functions to be in a Game Objects class this uses a pool of game objects that are also abstracted away so that the abstract script class is just appended to the game object, and called

Game objects

|

|

|- Array of GameObjects

|

|- Exposed functions to add new game object

|

|- exposed functions to get game objects (eg getAllOfTag([tag]))

These abstractions make it easy to compose game objects in future

input

There is no good way to track inputs, and key presses in OpenGL so I ended up making my own abstract version of the input system, I tried to mirror this to be similar to how it is done in other game engines This ended up being a kind of amalgamation of how Unity handles key inputs in the new input system, And how sfml / raylib handles key inputs, with just a ping if KeyDown is true I also had to add a mouse button tracker, which I decided to abstract by adding a way to quickly assign a button press to an array, and it is then handled and called as needed so you can just append an abstract function, and assign a mouse key, and it will be called (single line)

Planet

The main centre point of my project is the planet What i wanted for the planet is for it to be a fully explorable sphere the sphere to have things on it and for enemies to spawn on it that you can shoot then you proceed to the next planet after defeating a boss So the first thin i needed was the noise to deform the planet I originally started by making a Ico sphere and trying to apply noise to that This failed for one main reason. ICO spheres use subdivisions this means that after instantiation and subdivisions, subdivisions after 3 / 4 lag the game tremendously so after all the research and work that was put in to make the Ico sphere, I had to scrap it because it was too slow (this was around 2 weeks of work that was scrapped due to lag) so I did more research, then i stumbled on the cube sphere, which allows for very fine control of the vertex count on the sphere, and didn't have the issue of a normal sphere where the poles would distort movement

so i got to work on the sphere after creating it and hooking up the mesh i tried to implement the noise function I had for the flat ground tile this obviously didn't work so I proceeded to look up alternatives, I found a noise class available online which allows for the deformation to be even around the surface specifically allowing specific points to always be the same height, this got rid of the need to also remember previous positions as the deformation at specific locations is always the same this fixed the lag issue

I then needed the mesh to be coloured I used a simple height function, that looks for the highest point, and the lowest point, this is my 0 and 1 then every height in between can be of different colours eg 0 - 0.3 sand, 0.3 - 0.7 grass, 0.7 - 1.0 rock (mountains) this looked very good so i ended up keeping it, i added small blending factor between layers and that was the colours

Later I converted the colours to also accept textures at the certain heights

planet editor

I contemplated how to make the planet generation more interesting and came up with the idea of making a planet editor this proceeded and ended up taking roughly 3 weeks to fully implement correctly and reliably. the editor needed to edit the planet colour, and the noise settings (for now) I also needed a way to open and close this menu, and I didn't want to make it into a key bind (too basic in my opinion) so I created the chat system, this is very similar to many other games I also needed to track commands that are called and the arguments that they receive I decided to then abstract this and move it to a separate static class. This was to increase speed of development in future, so all i would need to do is add a single line to add a new command to the command interpreter. This included the /GUI command which opens the world editor (later added /godmode, / changename, /clear (clear chat), /connect /reconnect, etc, this was very easy to implement thanks to the abstractions)

Save / load

Json parser I thought of making a manual Json parser or parsing the planet into a simple text file, however after a quick bit of research io found Nhlomanns Json parser for C++ I decided to use it to speed up the process of creating new planets a bit

1. load planet from Json
2. save planet to Json
3. load random from Json

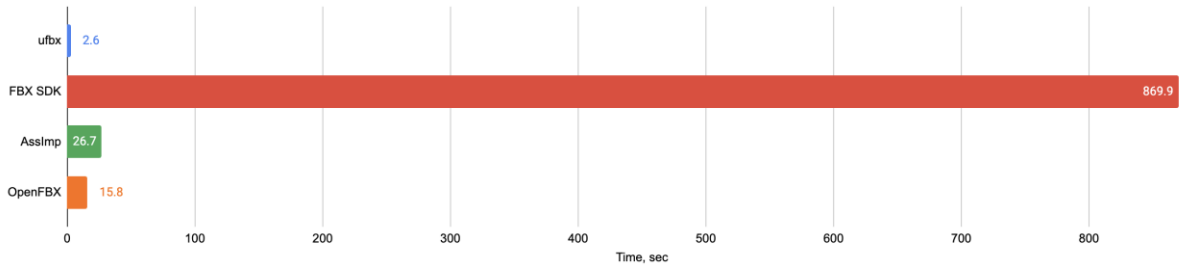
these are the main things that were implemented, this also had to get added to the planet editor for ease of use and creation after this implementation it meant that the creation of new planets could be done in the matter of minutes, and then saved for the user the user then loading a random planet to play on, making every play through unique

shader store

I added an abstract for shaders, I noticed every shader had similar code for instantiation and after learning that each shader needed to be reset for each object (different variables for objects, but use same shader) I realised that the shader can be very easily abstracted to a different file so i ended up making the shader store which stores all shader pairs (fragment, and vertex shader) this meant that allthe instantiation of the shaders could be abstracted from the normal files. Making them much simpler to read

model store

Loading of models from FBX files is very complicated due to the encryption of .fbx, and the storage being weird this left me a few options, using a premade library, or making my own i used the UFBX library which is by far the lightest one



This allowed me to load FBX files, but I still had to parse them manually into my render object after figuring all that out, I finally had models in my game

I then proceeded to make a model store, as it seemed that reloading the same model more than once caused errors, and crashes this means that every model is loaded and cached allowing for fast retrieval

texture store

Texture store functions similar to the FBX store where it stores the data for each image this makes it easy for the code to retrieve the same image for multiple different objects

particles

Particle pool and animated textures this uses a similar idea to the previous where all the code is abstracted away, and the spawning of particles only takes 1 single line this works well and allows for variants, and such to be used by the code, this can be seen in the muzzle flash, where different flashes are used for the muzzle

enemy abstract state

I used a simple command pattern to abstract the enemy pattern and to keep the code more simple this works similar to a finite state machine an abstract enemy state is given to the enemy on instantiation the enemy then uses this abstract state to move around, attack, idle, die etc

This allows for the fast creation of new enemies with unique states, so this was chosen as the way to handle enemies for that sole reason

this took a while to implement however as there were errors with the manager, and different errors with the enemy movement along the surface

model partner script

when grass instancing was created (explained next) there was an issue. each instance needed a seperate model however this was super inneficient and laggy so i tried to replicate the flyweight pattern using my own implementation (flyweight designed by myself, but after research i realised the code mimics fly weight) this functions similarly, and works very well and alleviates lag the model partner script exists to set object specific parameters this being positions, the model is a single instance and the partner script changes the position and such this is because the model can be heavy to instantiate on computation, this design worked and alleviated the lag, allowing for more instances for almost free (outside of cost of instancing itself)

surface instancing

Surface instancing took a long time to figure out, this is because the data for the planet isnt really stored anywhere outside of the vbo, this is okay for the planet rendering but to place an instance on the mesh would require a lot of work for the pc to sample all vertices so i found another way of doing it by sampling the noise from the original noise class this worked very well and allowed me to disperse objects across the mesh randomly

weapons

weapon creation is also abstracted to a high level allowing for very fast creation of new weapons, the only things that are needed are the data (full auto, mag size, reload time etc.) and a gun model & texture then by simply adding to the array of weapons, it is added as a useable weapon this makes it very simple to make new weapons, and makes it very easy to make a weapon editor (if it were saved to Json and such) this was done for future expansion, but has not yet been implemented.

interaction

using the tag system i have added an interaction method, this simply uses a hit sphere and ray calculation after that it sends a pick up method (caught by a script) for it to destroy, this is easily expandable thanks to the sendMessage function, It allows for the creation of new items that can be on the ground / interacted with, this is currently used for just collecting rocks, and has not been expanded much past that, this is due to lack of time, I would in future use this for opening doors, picking up new weapons etc.

enemy

the enemy who uses the enemy state machine has an interesting attack pattern, this was created the way it was thanks to testing now it highlights white and has a blood splatter particle whenever it is shot, this clearly shows hit effect

the slime enemy has an interesting attack (tested for fun sake) when it gets close it enters the attack state it stands still for 0.7 seconds (most balanced time) it then turns red 0.1 seconds before charging, if the player doesn't move outside of radius in that 0.1 seconds (rewarding for very fast reaction players) it charges at the player, this charge lasts 1 second (so it is possible to avoid if you are fast to sprint out of the way) and this damages the player if it hits the player (red during entire charge attack) it then waits 3 seconds stationary before it tries to charge again

this sounds complicated, but is very normal when the player is playing, and very clear for the user it also rewards players that are paying attention (to the red glow) it is also fair as the player has some time to get away after avoiding / getting hit

main menu

the main menu was specifically made to be simple and hopefully if needed expandable I used ImGui because I already knew how it worked and had it implemented elsewhere in the repo This also meant I didn't have to overcomplicate the code because it is instanced, and updates live this all meant that the main menu could be reliable and useable

screen overlay

The screen overlay is used for 3 things, taking damage overlay (red flash) fade in and out of the level and underwater overlay the underwater overlay is there to mimic how other games handle underwater, making it more believable as it is harder to see underwater I used a simple quad that is displayed in the game view, and overlays over everything, this gave the effect I wanted (I wanted it to be clearly visible when you are damaged)